Alerting and Monitoring Standards and Practices

Overview

The purpose of this document is to describe the standards and practices that ITSE, scrum masters, product owners, and developers use to facilitate alerting and monitoring across UMPDE. This document is meant to evolve as our standards and practices do.

When to Alert

⚠ **Avoid unactionable alerts!**

Alerts are not just informational; they are meant to be acted upon.

A good rule of thumb is to ask yourself *"should an action occur based on some state of the system or application?"* then an alert should be created.

Examples:

1. Scaling
2. Users need to be notified
3. ITSE needs to be notified
4. A new user story should be created in the backlog to enhance a feature or fix a defect
5. Developers need to know when a performance issue is occurring in production so it can be monitored

Who to Alert

The development team should create an action group or email distribution for team alerts.

Alerts for the ITSE audience should be sent to UMSE-TechOps@evicore.com.

Alerting Refinement

Alerting and monitoring must be maintained just as any other piece of software that we build. Requests for change in alerting and monitoring should follow the current scrum processes that teams use today to make change requests.

ITSE, product owners, and developers should work together to

1. Optimize alerts by adjusting thresholds

2. Reduce false positives
3. Define new alerts

Implications to Definition of Done

Required alerting must be implemented to consider a user story '**done'** or an application as '**production ready'**. It is not an optional or nice to have feature.

It is understood that this may be an incremental process as many applications today do not have required alerting implemented.

> ⚠ It is very important to reduce noise and false positives. While it may seem that these issues do not affect anyone, they can actually end up costing the company thousands of dollars when production issues occur causing red herrings for support teams.

Exceptions

If a required alert is not applicable, then the expectation should be documented and examined to help evaluate if the alert should be required or recommended.

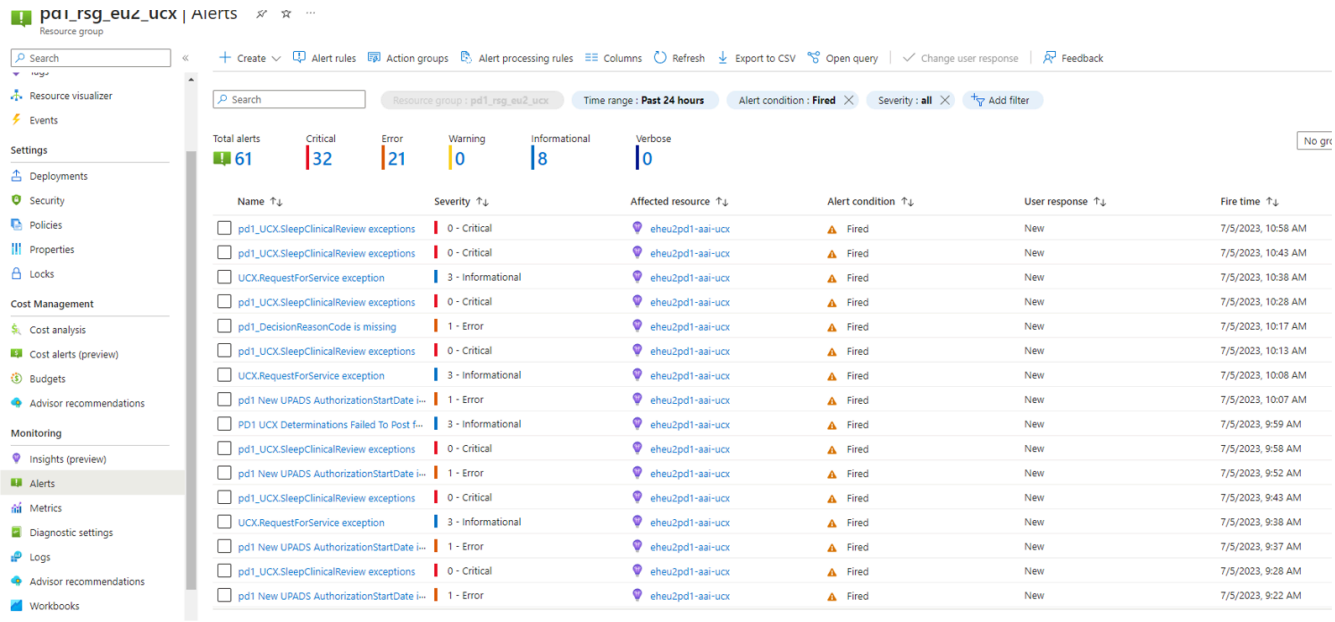
Tools

Application Insights

Application Insights is an extension of Azure Monitor that we use to monitor and alert applications at eviCore.

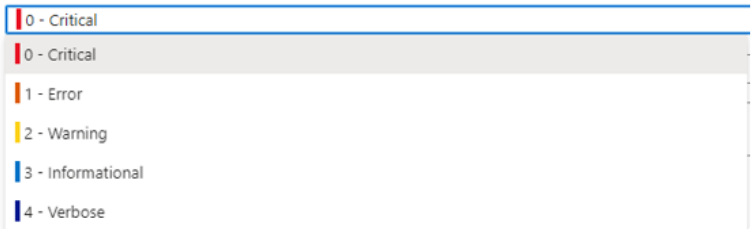
Managing Alerts with the Azure Portal

Alerts for eviCore products are viewed and managed by ITSE and Product owners through the Azure Portal. Alerts are meant to be acted upon so the intent is to keep the portal manageable by reducing noise or unwanted information so that ITSE, Product Owners, or developers can quickly make sense of what they are seeing.



Alert Classifications

Azure allows for 4 severity types to classify alerts.



Name	Severity	Description
Critical	0	Loss of service or application availability or severe degradation of performance. Requires immediate attention
Error	1	Degradation of performance or loss of availability of some aspect of an application or service. Requires attention but not immediate.
Warning	2	A problem that doesn't include any current loss in availability or performance,

		although it has the potential to lead to more severe problems if unaddressed.
Informational	3	Doesn't indicate a problem but provides interesting information to an operator, such as successful completion of a regular process.
Verbose	4	Doesn't indicate a problem but provides detailed information that is verbose.

Required and Recommended Alerts

Development teams provision their own cloud resources using infrastructure as code and are responsible for alerting and monitoring those resources. It is *recommended* to use shared infrastructure as code modules (Terraform, Bicep) that are already configured with these requirements.

Cosmos DB Containers

Rule	Severity	Audience	
99% AVG Normalized RU Consumption	Critical	Development Team, ITSE	Required
80% AVG Normalized RU Consumption	Recommended	Development Team	Recommended

App Service

Rule	Severity	Audience	
Availability < 98%	Critical	Development Team, ITSE	Required

CPU Percentage > 80%	Critical	Development Team, ITSE	Required
Web Server Memory > 90%	Critical	Development Team, ITSE	Required

Kafka

Rule	Severity	Audience	
Stuck Observer	Critical	Development Team, ITSE	Required
Message Dead Lettered	Error	Development Team	Required
Consumer Circuit Breaker Tripped	Critical	Development Team, ITSE	Required

Redis

Rule	Severity	Audience	
Memory > 97%	Critical	Development Team, ITSE	Required
CPU > 97%	Critical	Development Team, ITSE	Required
Connections > Threshold	Error	Development Team	Required

Web Service Alerts

Rule	Severity	Audience	
API Circuit Breaker Tripped	Critical	Development Team, ITSE	Required
API Performance < SLA	Critical	Development Team, ITSE	Required

Web Response 500 > Threshold within 10 mins	Critical	Development Team, ITSE	Required
Prolonged external API failure such as UPADS, Object Valet	Critical	Development Team, ITSE	Required
Health Check Failure	Critical	Development, ITSE	Required

> Some services do not have an explicit SLA and it is understood that the team will need to agree upon an acceptable SLA that should be used for alerts

On-Prem SQL Server

Rule	Severity	Audience
Row Dead Locking	Critical	Development Team, ITSE, DBA Team, Command Center

Roadmap

Short Term

The initial phase will be to leverage current tools and practices to improve alerting and monitoring. The primary motivator is to reach a baseline that all teams are using to improve MTTR and standardize communication with ITSE.

Goals:

1. Improve communication time with ITSE
2. Improve required alerting
3. Decrease alerting noise to make alerts more reliable

Medium Term

After the short term goals have been met, we will investigate efficiencies that could be gained by leveraging a new pattern for alerting based on the subscription pattern and alarm funnel.

P2P (path to production) requires a WAR (well architected review). Part of the requirement is to ensure that alerts are configured. There is a desire for both Azure and AWS to go through the same alerting mechanism.

Long Term

The long term goal is to investigate different tooling and processes that improve observability and efficiency.

- Investigate using Dynatrace as it is already procured product that is approved by Cigna.
- Investigate Cribl, Splunk, and MattersMost

Related Articles

eviCore Architecuture - Monitoring and Alerting

Technical Excellence - Evicore Platform, Alerting and Monitoring

Azure Monitor best practices: Alerts and automated actions - Azure Monitor | Microsoft Learn

Terraform Modules in Azure Devops

Azure Monitor Alerts - Cloud COE (42) - Path to Production (FMEA)